Lab 1 – Windows IT Support & Troubleshooting Home Lab

Objective

The objective of this lab was to build a Windows-based IT support environment in a virtual machine and practice common help desk tasks, including system configuration, user and permission management, network troubleshooting, command-line tools, system monitoring, and troubleshooting documentation.

Environment

- **Host Operating System:** Windows 10 Home
- **Virtualization Platform:** Oracle VirtualBox
- **Guest Operating System:** Windows 10 Pro
- **VM Memory:** 4 GB RAM
- **VM Processor:** 2 CPU cores
- **VM Storage:** 60 GB virtual disk
- **User Accounts:** LabAdmin and HelpDeskUser

Tools Used

- Command Prompt (CMD)
- PowerShell
- Event Viewer
- Windows Control Panel
- Local User Management
- Windows Network Configuration

1. Windows Installation & System Configuration

Windows Virtual Machine Setup

I created a Windows 10 Pro virtual machine in Oracle VirtualBox to provide an isolated environment for practicing IT support and troubleshooting tasks.

The virtual machine was configured with 4 GB of RAM, 2 CPU cores, and a 60 GB dynamically allocated virtual hard disk. Windows 10 Pro was installed using a clean installation.

Local Administrator Account

During the Windows setup, I created a local administrator account named **LabAdmin**. This account was used for administrative configuration and troubleshooting tasks throughout the lab.

System Verification

After installation, I verified the Windows version and system configuration using the `winver` utility and Windows System Information.

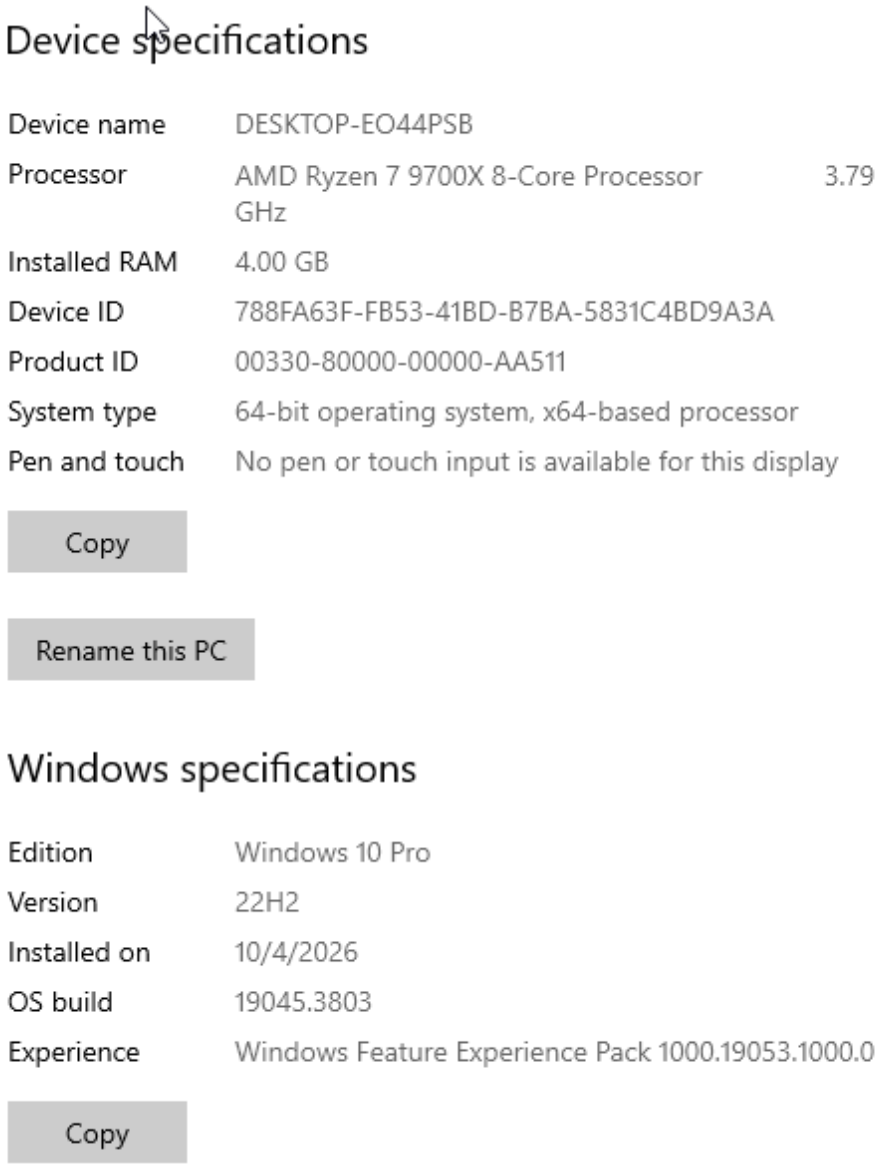
This confirmed that the virtual machine was running Windows 10 Pro and that the expected processor, memory, and system architecture were recognized.

Evidence:

- `01_Windows_Version.png`



- 02_System_Information.png



Device specifications

Device name	DESKTOP-EO44PSB	
Processor	AMD Ryzen 7 9700X 8-Core Processor	3.79 GHz
Installed RAM	4.00 GB	
Device ID	788FA63F-FB53-41BD-B7BA-5831C4BD9A3A	
Product ID	00330-80000-00000-AA511	
System type	64-bit operating system, x64-based processor	
Pen and touch	No pen or touch input is available for this display	

Copy

Rename this PC

Windows specifications

Edition	Windows 10 Pro
Version	22H2
Installed on	10/4/2026
OS build	19045.3803
Experience	Windows Feature Experience Pack 1000.19053.1000.0

Copy

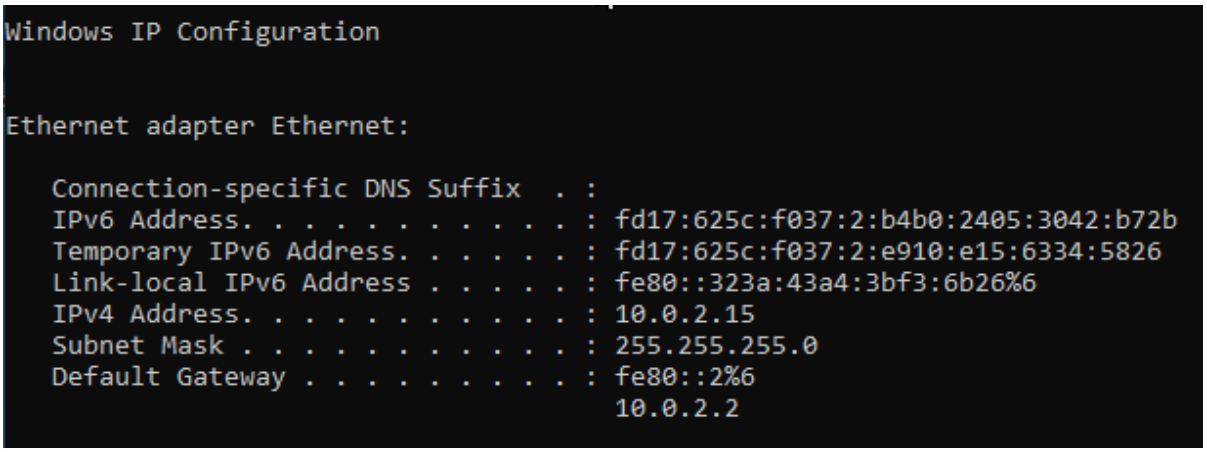
2. Network Configuration & Connectivity Testing

IP Configuration

I used Command Prompt and the `ipconfig` command to inspect the virtual machine's network configuration. This allowed me to view the assigned IP address, subnet information, default gateway, and other network configuration details.

Evidence:

- `03_IP_Configuration.png`



Network Connectivity Testing

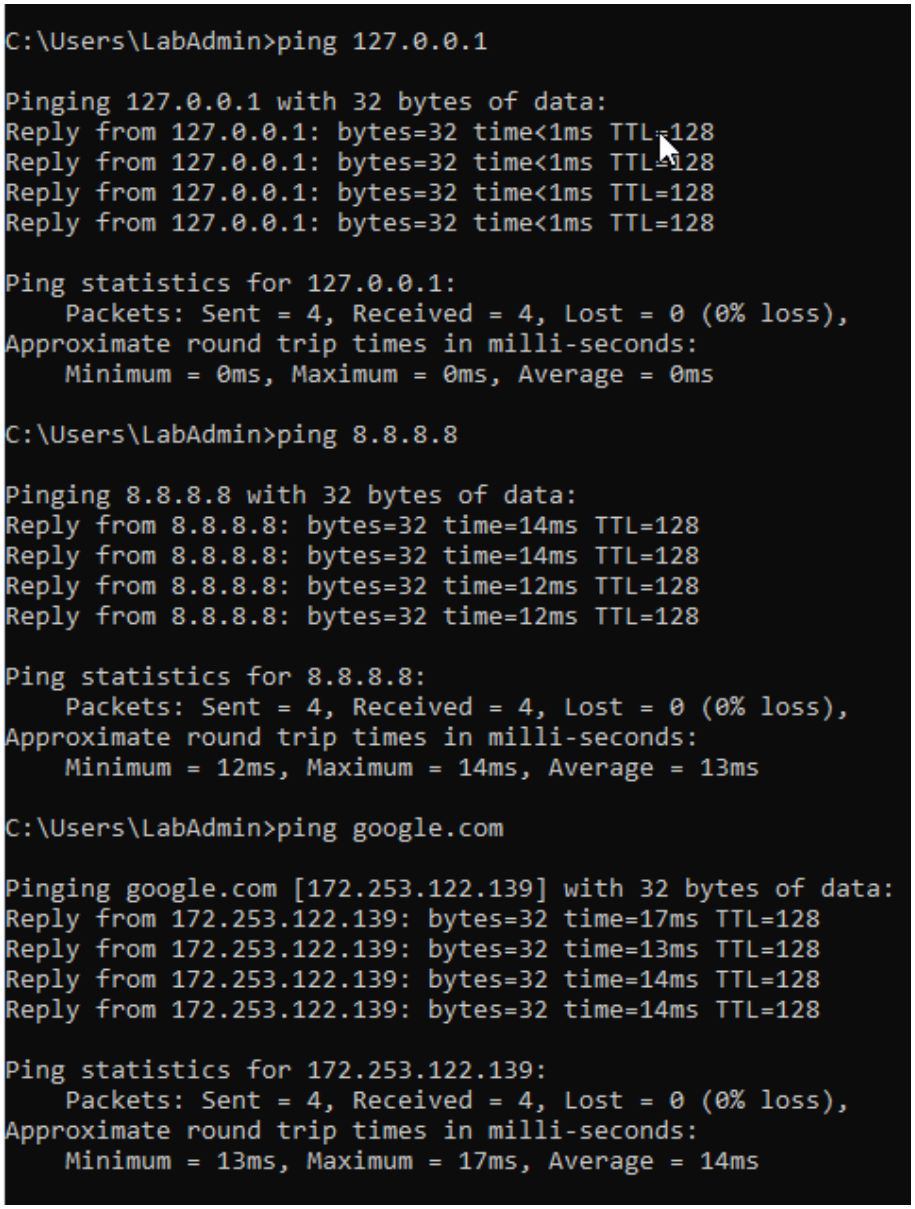
I used the `ping` command to test connectivity at multiple levels:

- `ping 127.0.0.1` — tested the local TCP/IP stack.
- `ping 8.8.8.8` — tested connectivity to an external IP address without relying on DNS.
- `ping google.com` — tested external connectivity using a domain name, which also tests DNS name resolution.

All three tests were successful, confirming that the virtual machine's network connection and DNS resolution were functioning correctly at this stage.

Evidence:

- 04_Network_Connectivity.png



What I Learned

The `ping` command is useful for narrowing down network problems. Testing an IP address separately from a domain name can help determine whether a problem is related to general network connectivity or DNS resolution.

3. User Accounts, UAC & Permissions

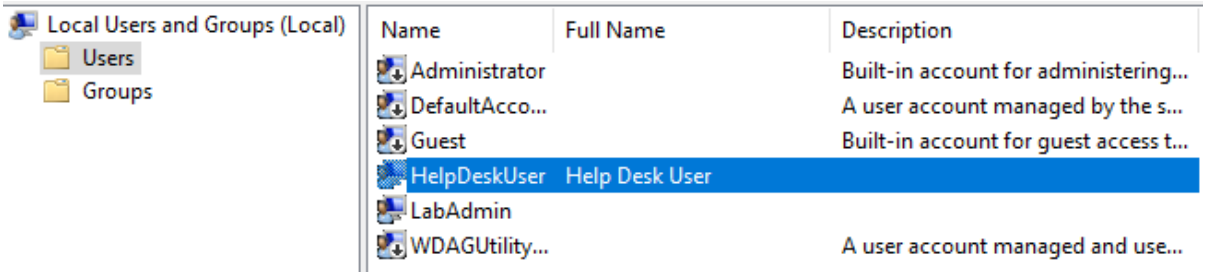
Creating a Standard User

I created a local user account named **HelpDeskUser** using Local Users and Groups. The account was configured as a standard user rather than an administrator.

This provided a separate account for testing how Windows handles tasks that require elevated administrative privileges.

Evidence:

- `05_User_Account_Creation.png`



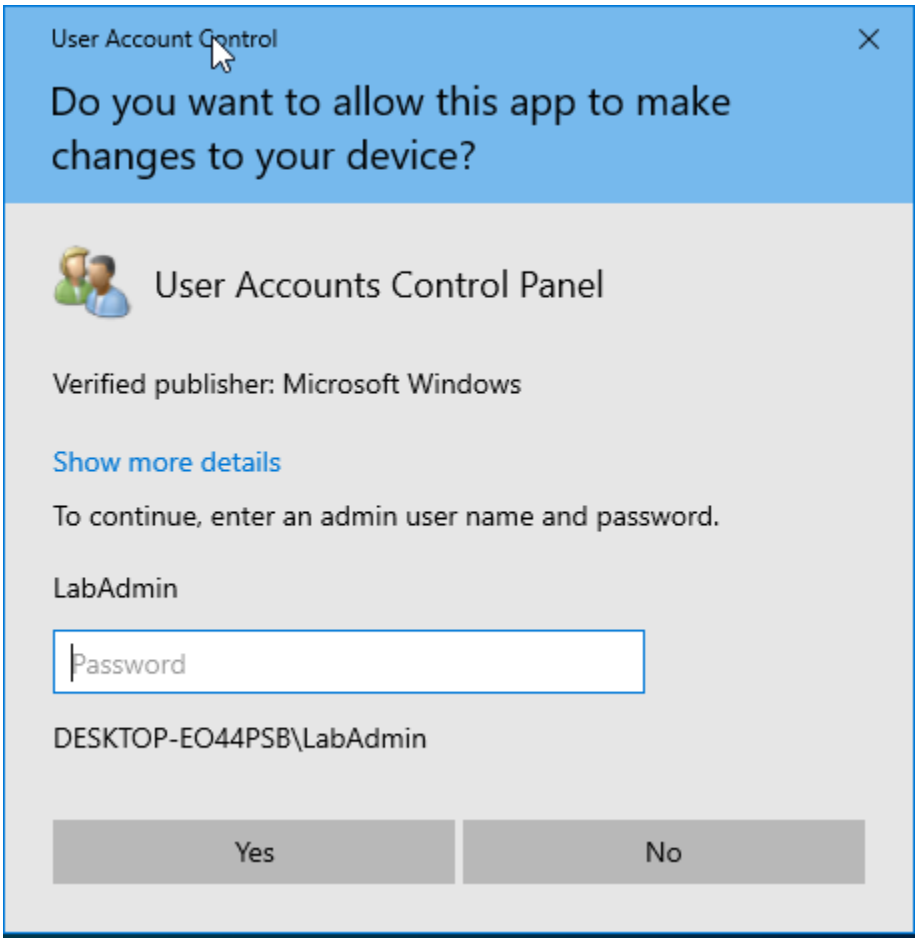
Testing Standard User Restrictions

I logged into Windows using the **HelpDeskUser** account and attempted to manage another user account. Windows prompted for administrator credentials from the **LabAdmin** account.

This demonstrated **User Account Control (UAC)** and the principle of least privilege. Standard users can perform normal tasks but require administrator authorization for certain system-level changes.

Evidence:

- `06_Standard_User_UAC.png`



File Sharing & Permissions

I created a folder named `C:\IT_Lab_Shared` and configured it as a network share named **IT_Lab_Shared**.

The share permissions were configured so that **Everyone** had Read access, while Change and Full Control were not enabled.

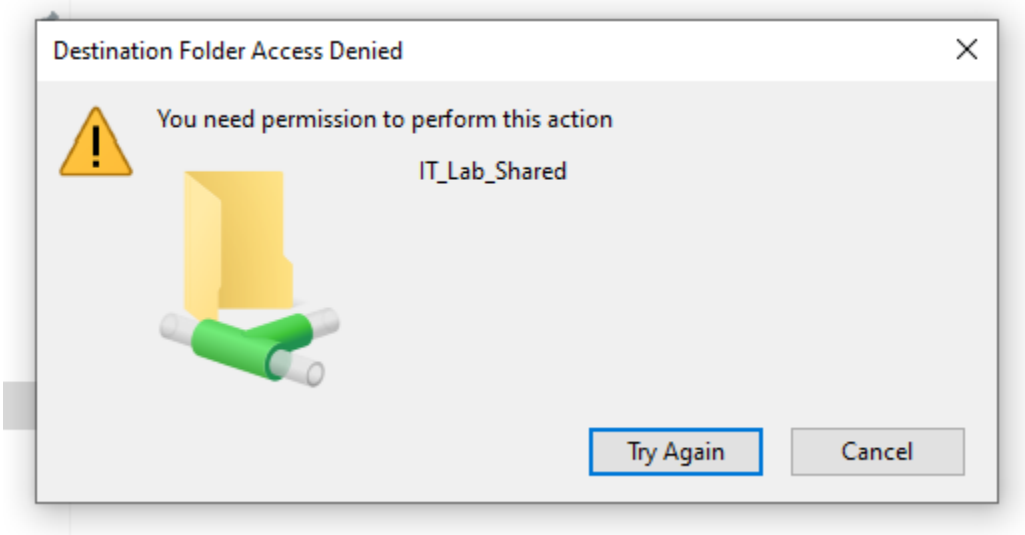
I then tested access through the network path:

\\localhost\IT_Lab_Shared

An attempt to create a file through the network share was denied, demonstrating how permissions can restrict what users are allowed to do.

Evidence:

- `07_Share_Permissions_Access_Denied.png`



What I Learned

Windows permissions help control what users can access and modify. A standard user does not automatically have administrative privileges, and UAC can require administrator authorization when a task needs elevated permissions.

I also learned that local access to a folder and access through a network share can be affected by different permission settings.

4. Command Line & PowerShell

System Information with Command Prompt

I opened an elevated Command Prompt and used the `systeminfo` command to retrieve detailed information about the Windows system.

This provided information such as the operating system version, system configuration, processor, memory, and Windows installation details.

System Information with PowerShell

I opened an elevated PowerShell session and used:

```
Get-ComputerInfo
```

This command provided detailed computer and operating system information through PowerShell.

Local User Management with PowerShell

I used:

```
Get-LocalUser
```

to view the local user accounts configured on the Windows system.

The command confirmed the presence of both **LabAdmin** and **HelpDeskUser**.

Process Management

I used:

```
Get-Process
```

to view currently running processes.

I then used:

```
Get-Process | Sort-Object CPU -Descending | Select-Object -First 10
```

to sort processes by accumulated CPU time and display the top 10 results.

Resource Monitoring

I used:

```
Get-Counter '\Processor(_Total)\% Processor Time'
```

to measure overall CPU utilization.

I also used:

```
Get-Counter '\Memory\% Committed Bytes In Use'
```

to check memory utilization.

These commands provided a way to investigate system performance from the command line.

What I Learned

I learned that Command Prompt and PowerShell provide powerful tools for gathering system information, managing users, viewing processes, and investigating system performance.

I also learned that the CPU value shown by `Get-Process` represents accumulated processor time for a process rather than its current CPU percentage. `Get-Counter` can be used to obtain current performance-counter measurements.

5. Event Viewer & Troubleshooting

Event Viewer

I opened Windows Event Viewer using `eventvwr.msc` and inspected the **Windows Logs → System** log.

I reviewed system events, including a Kernel-General event, to practice identifying and interpreting Windows system activity.

This provided experience with using Event Viewer as a troubleshooting tool for investigating system behavior and potential problems.

Simulated DNS Failure

To practice troubleshooting a realistic network issue, I intentionally changed the virtual machine's IPv4 DNS configuration to an invalid DNS server address.

I then tested the connection using:

```
ping 8.8.8.8
```

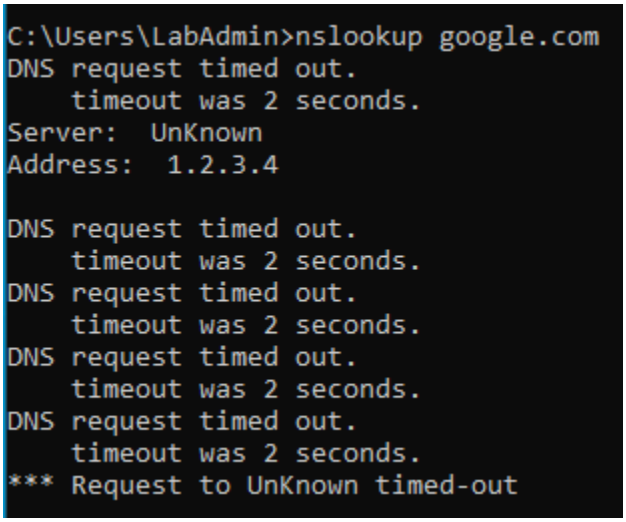
and:

```
nslookup google.com
```

The external IP address remained reachable, while `nslookup` timed out. This indicated that general network connectivity was functioning but DNS resolution was not.

Evidence:

- `08_DNS_Troubleshooting.png`



Resolving the DNS Failure

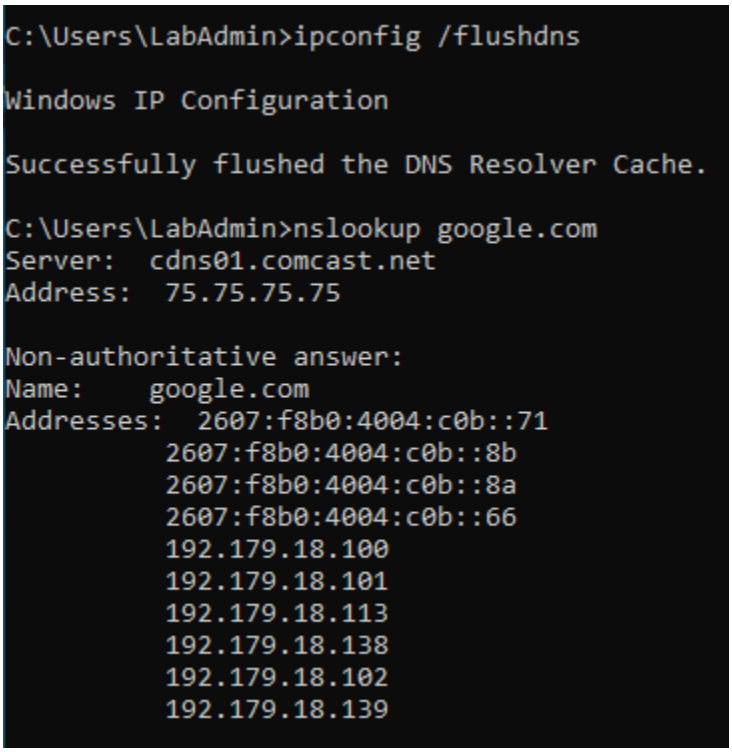
I restored the DNS configuration to **Obtain DNS server address automatically** and flushed the DNS cache using:

```
ipconfig /flushdns
```

I then used `nslookup google.com` and `ping google.com` to verify that DNS resolution and network connectivity had been restored.

Evidence:

- 09_DNS_Fixed_Verification.png



Troubleshooting Process

This exercise demonstrated a basic troubleshooting methodology:

1. Identify the symptom.
2. Test different parts of the connection.
3. Narrow down the likely cause.
4. Make a corrective change.
5. Test again to verify the solution.

The DNS exercise showed why troubleshooting should be performed systematically rather than immediately changing multiple settings at once.

6. Conclusion

This lab provided hands-on practice with common Windows IT support and troubleshooting tasks in an isolated virtual environment. I configured Windows, created and managed user accounts, worked with permissions, tested network connectivity, used Command Prompt and PowerShell, reviewed system events, monitored system resources, and diagnosed and resolved a simulated DNS issue.

Skills Demonstrated

- Windows 10 administration and configuration
- Virtual machine setup and management
- Local user account management
- User Account Control (UAC)
- File and network share permissions
- IP configuration and network troubleshooting
- DNS troubleshooting
- Command Prompt and PowerShell
- System and performance monitoring
- Windows Event Viewer
- Basic troubleshooting methodology
- Technical documentation